Giliojo mokymosi laboratorinis darbas Nr. 2

Konvoliuciniai neuroniniai tinklai

Rytis Šlefendoras | MGDMI - 3/2

1 dalis – Konvoliuciniai tinklai su LD2 duomenų rinkiniu

1.1 Duomenų rinkinio paruošimas

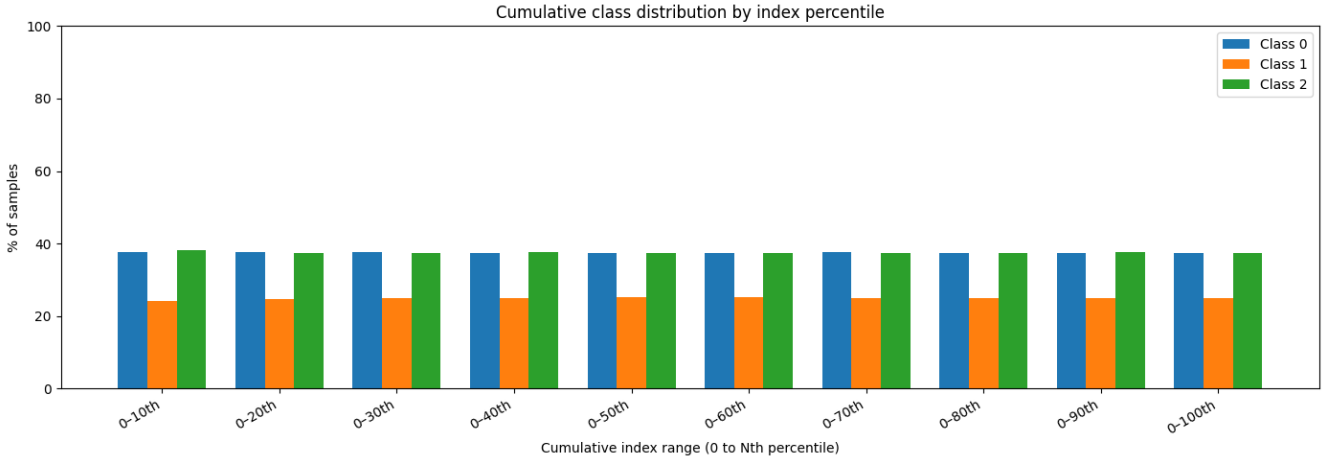
Darbui naudotas **LD2 duomenų rinkinys** – pilkų atspalvių (grayscale) 28×28 pikselių vaizdai, paimti iš 10 pradinių klasių. Pagal užduoties variantą, klasės 5 ir 8 buvo pašalintos, o likusios 8 klasės sugrupuotos į **3 jungtines klases**:

Nauja klasė	Originalios klasės
0	{0, 2, 4}
1	{3, 6}
2	{1, 7, 9}

Duomenys suskirstyti į tris aibes naudojant stratifikuotą padalijimą, siekiant išlaikyti klasių pasiskirstymą:

Aibė	Dalis
Apmokymo	~80%
Validavimo	~10%
Testavimo	~10%

Pastaba dėl **klasių disbalanso**: klasė 1 turi mažiau pavyzdžių nei klasės 0 ir 2 (testuojant: klasė 0 – 2100, klasė 1 – 1400, klasė 2 – 2100 pavyzdžių). Testuota tam tikrame duomenų procentilyje.



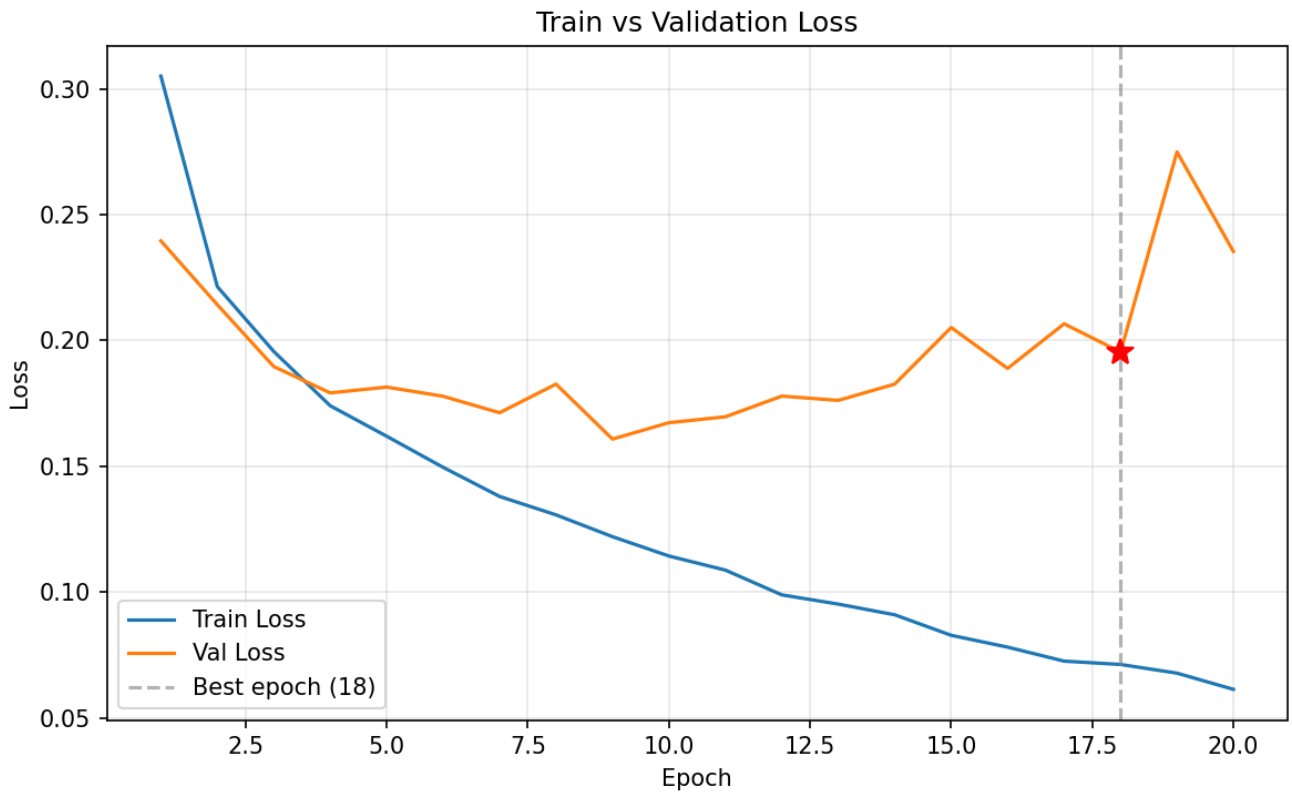
Dėl šios priežasties modelių vertinimui naudojamas **F1-makro** rodiklis, kuris vienodai atsižvelgia į visų klasių rezultatus nepaisant jų dydžio skirtumo.

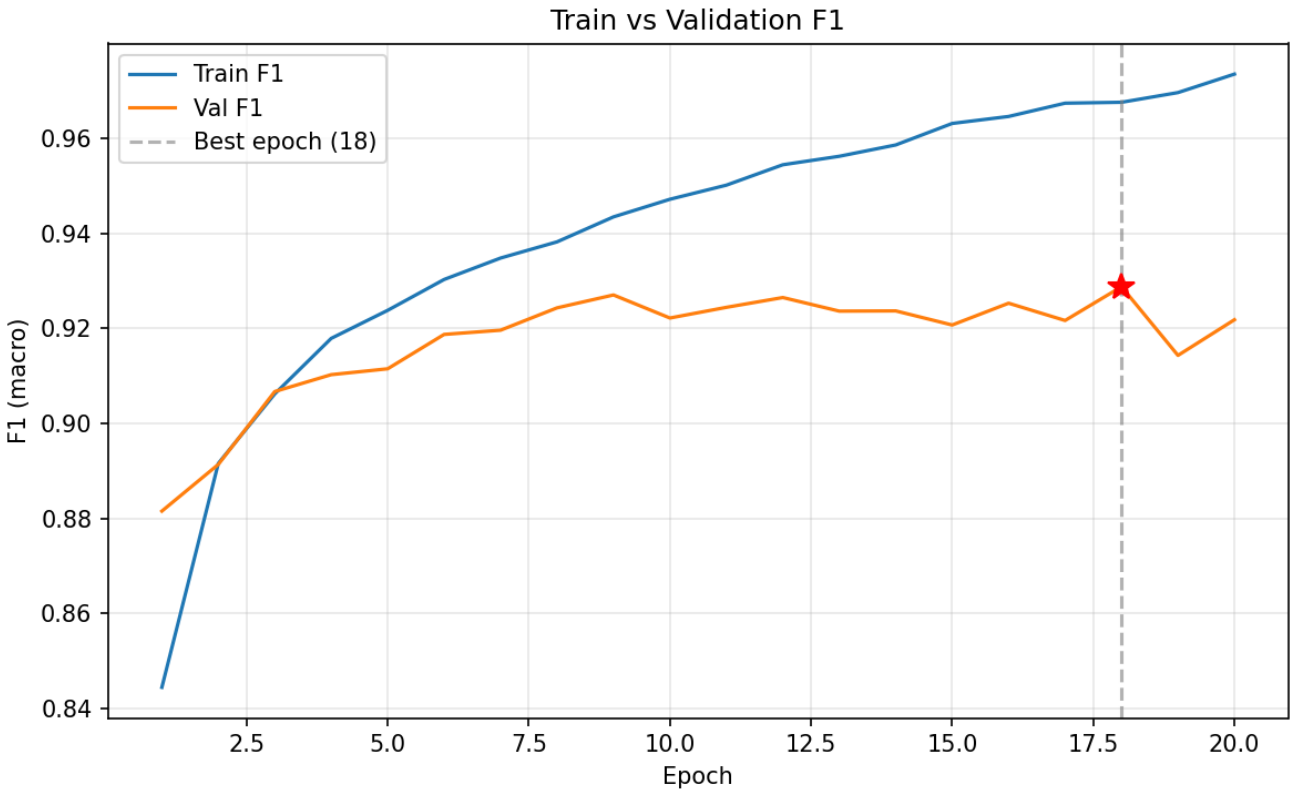
1.2 Architektūrų aprašymas ir apmokymo rezultatai

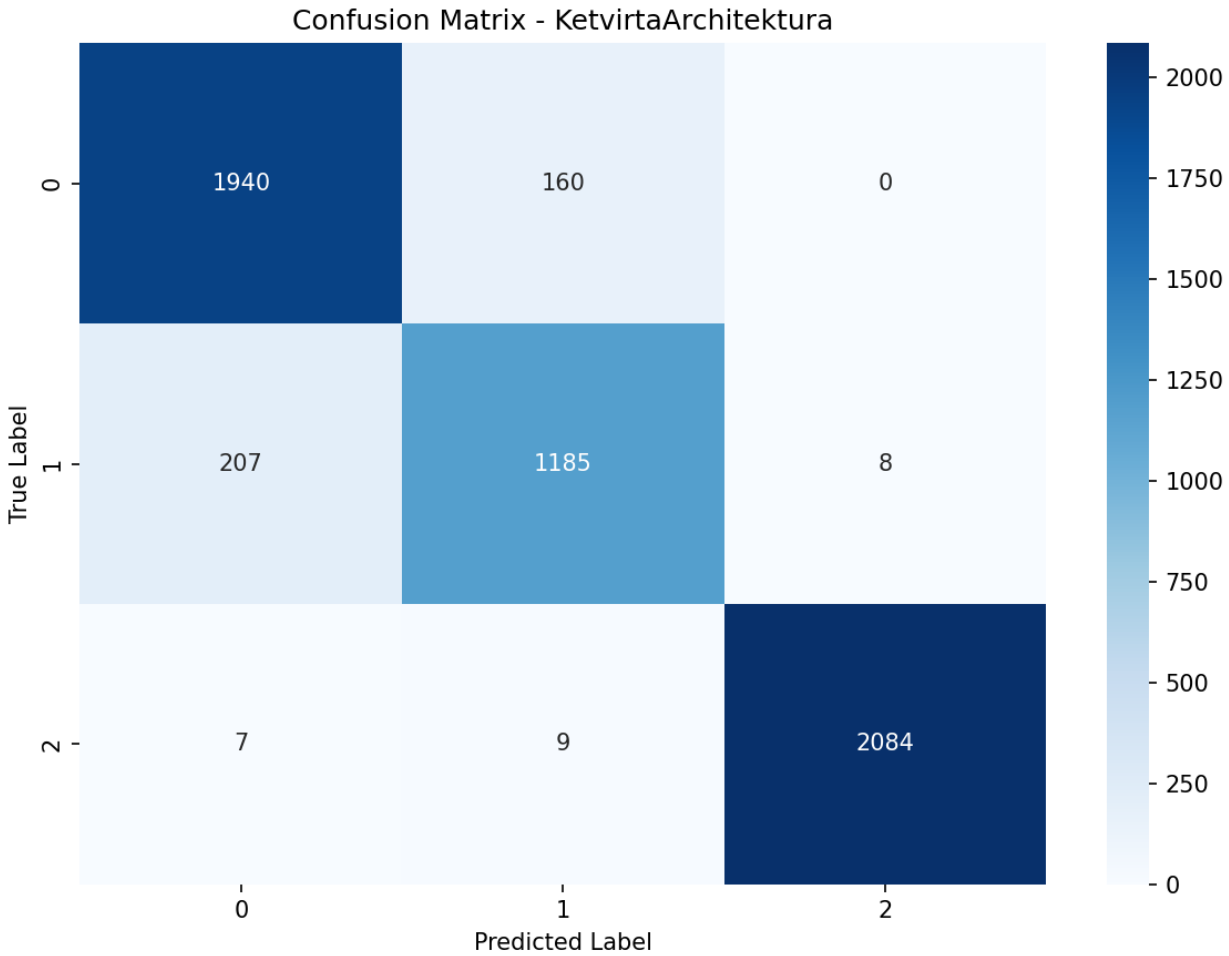
Išbandytos **keturios konvoliucinių tinklų architektūros**. Visos architektūros priima 28×28×1 įvesties vaizdus ir sprendžia 3 klasių klasifikavimo uždavinį. Apmokymas: 20 epochų, batch dydis 32, mokymosi greitis 0,001, optimizatorius Adam.

KetvirtaArchitektura Trys konvoliuciniai blokai su MaxPooling: Conv(1→32) → MaxPool → Conv(32→64) → MaxPool → Conv(64→128). Po konvoliucijų – du pilnai susieti sluoksniai (1152→64→3).

Apmokymo eiga:




Train vs Validation F1
Train F1
Val F1
Best epoch (18)
F1 (macro)
0.96
0.94
0.92
0.90
0.88
0.86
0.84
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0
Epoch



Geriausia epocha: **18**. Validavimo loss pradeda augti po 17-os epochos, rodydami pradžią persimokymui.

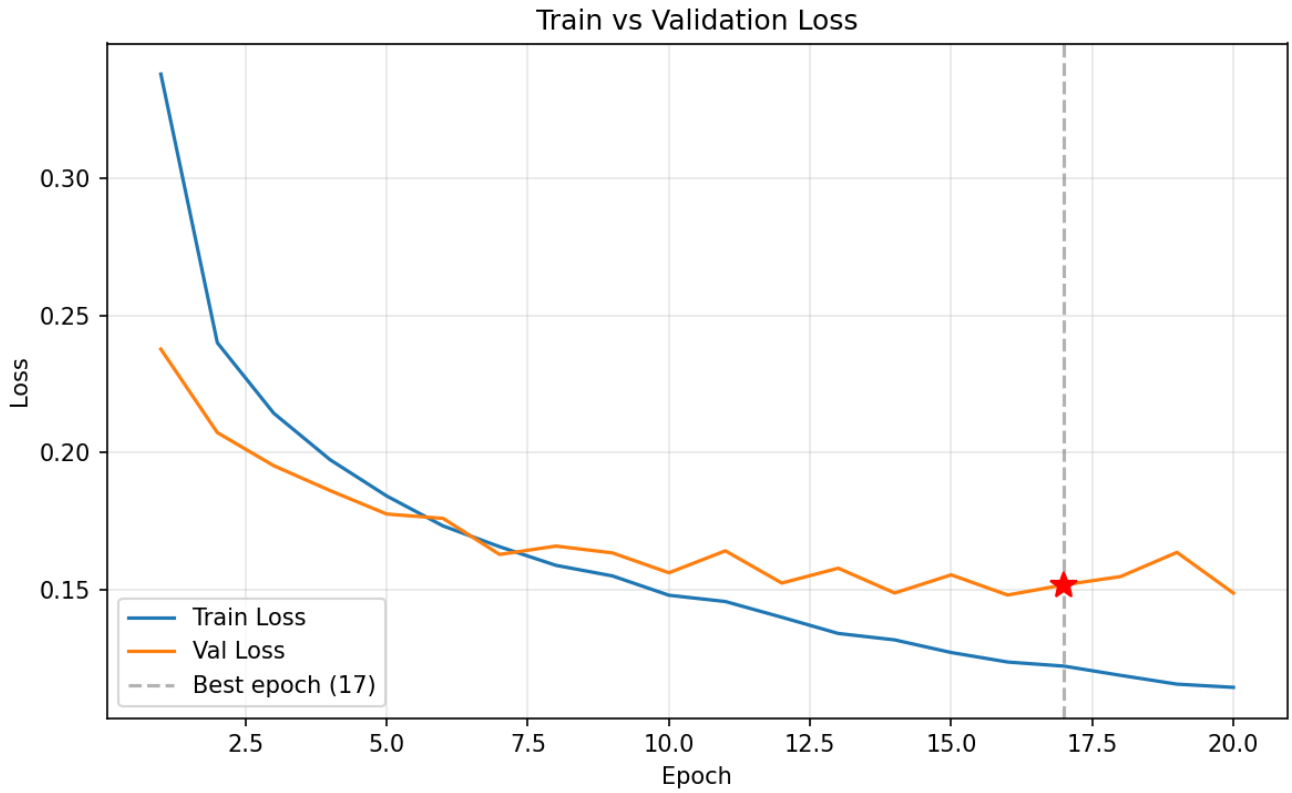
Testavimo rezultatai:

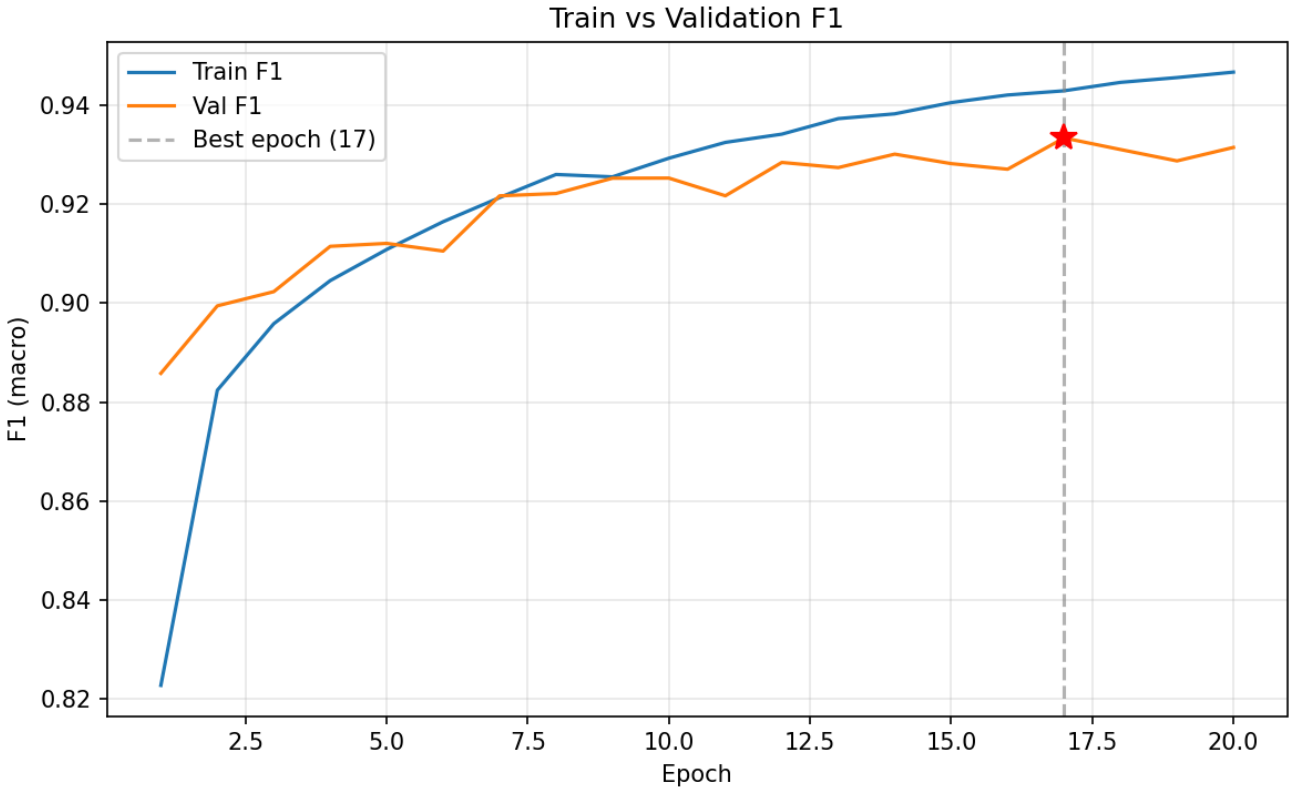
Klasė	Precision	Recall	F1
0	0.901	0.924	0.912
1	0.875	0.846	0.861
2	0.996	0.992	0.994
Makro vidurkis	**0.924**	**0.921**	**0.922**

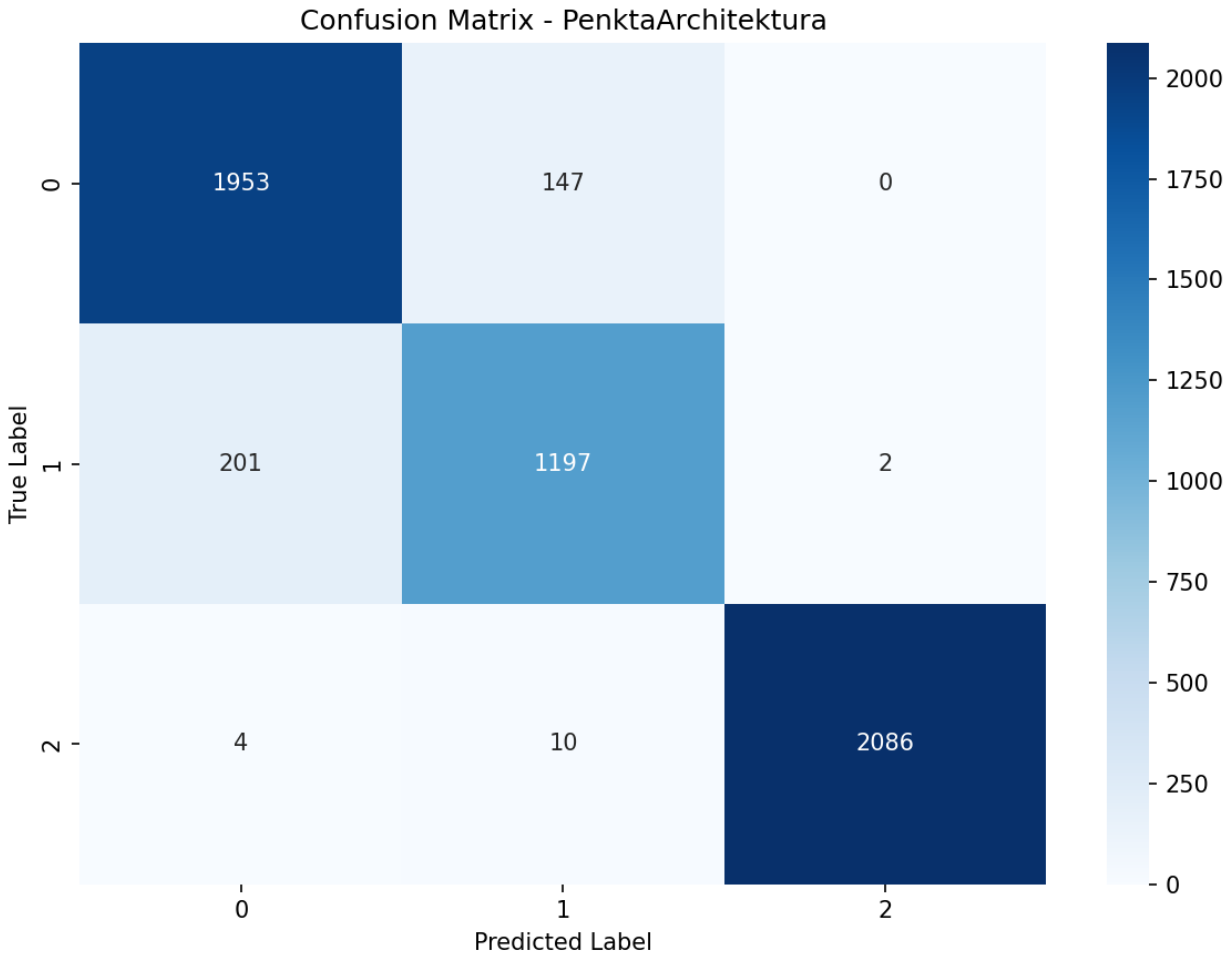
Bendras tikslumas: 93.02%

PenktaArchitektura Analogiška KetvirtaArchitektura, tačiau antrame bloke **MaxPool pakeistas AvgPool**, o prieš pilnai susietą sluoksnį įterptas **Dropout(0.4)** reguliarizacijai.

Apmokymo eiga:


Train vs Validation Loss
Loss
Epoch
0.30
0.25
0.20
0.15
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0
Train Loss
Val Loss
Best epoch (17)


Train vs Validation F1
F1 (macro)
Epoch
0.94
0.92
0.90
0.88
0.86
0.84
0.82
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0
Train F1
Val F1
Best epoch (17)



Geriausia epocha: **17**. Dropout šiek tiek sulėtino mokymosi eigą, tačiau pagerino generalizavimą lyginant su KetvirtaArchitektura.

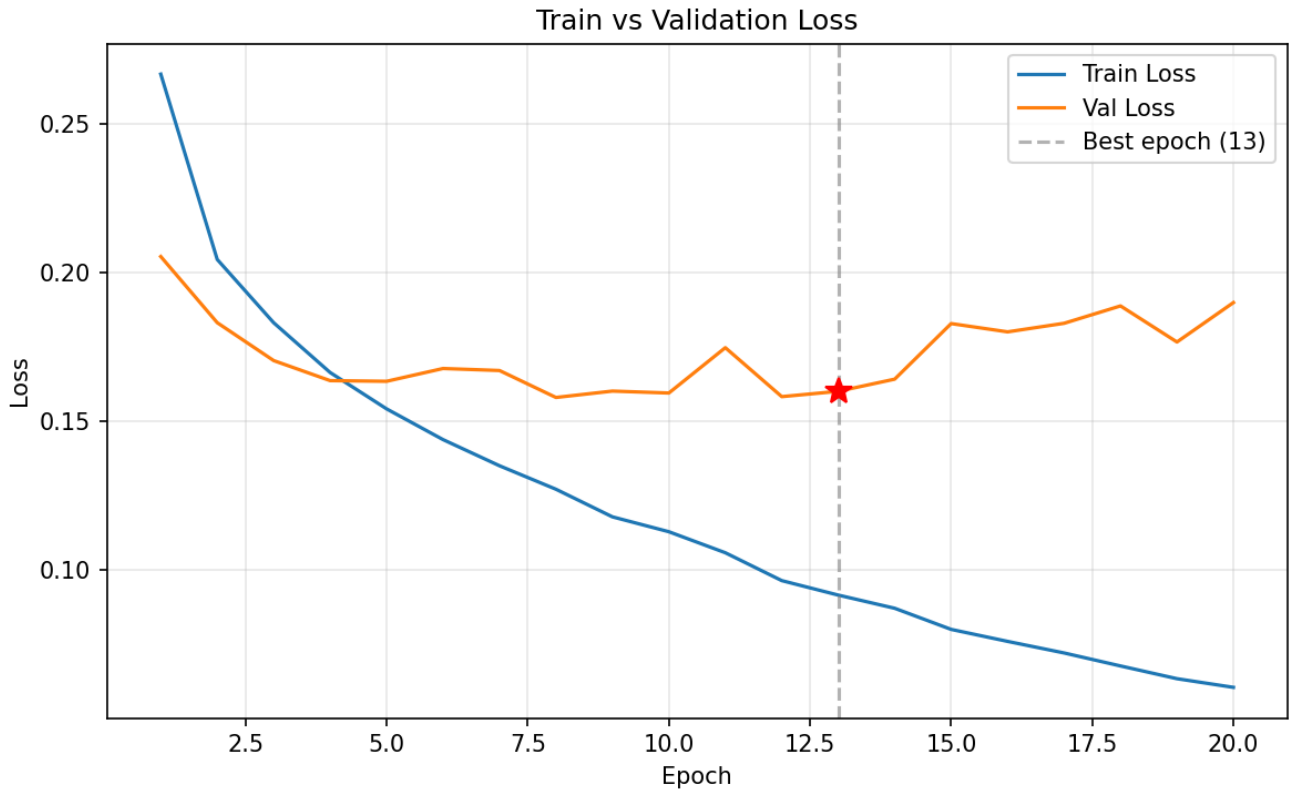
Testavimo rezultatai:

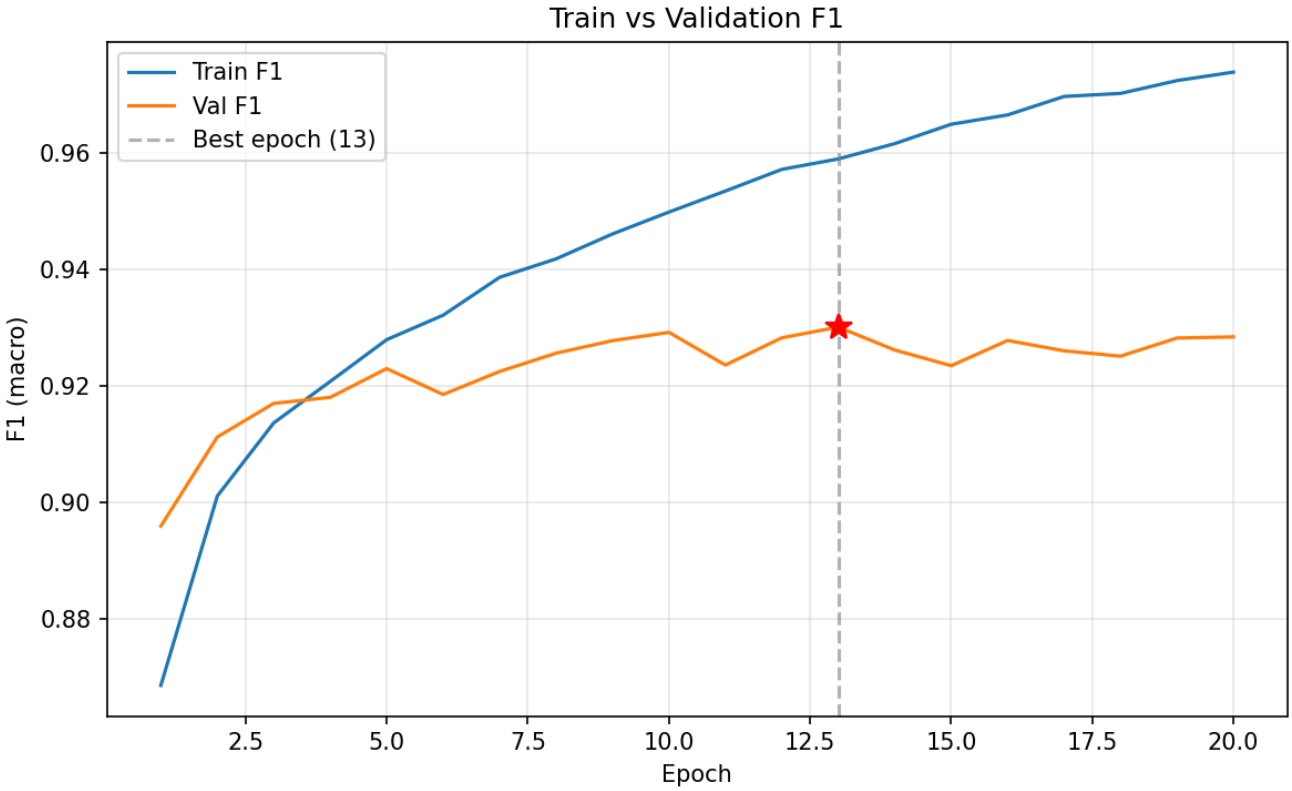
Klasė	Precision	Recall	F1
0	0.905	0.930	0.917
1	0.884	0.855	0.869
2	0.999	0.993	0.996
Makro vidurkis	**0.929**	**0.926**	**0.928**

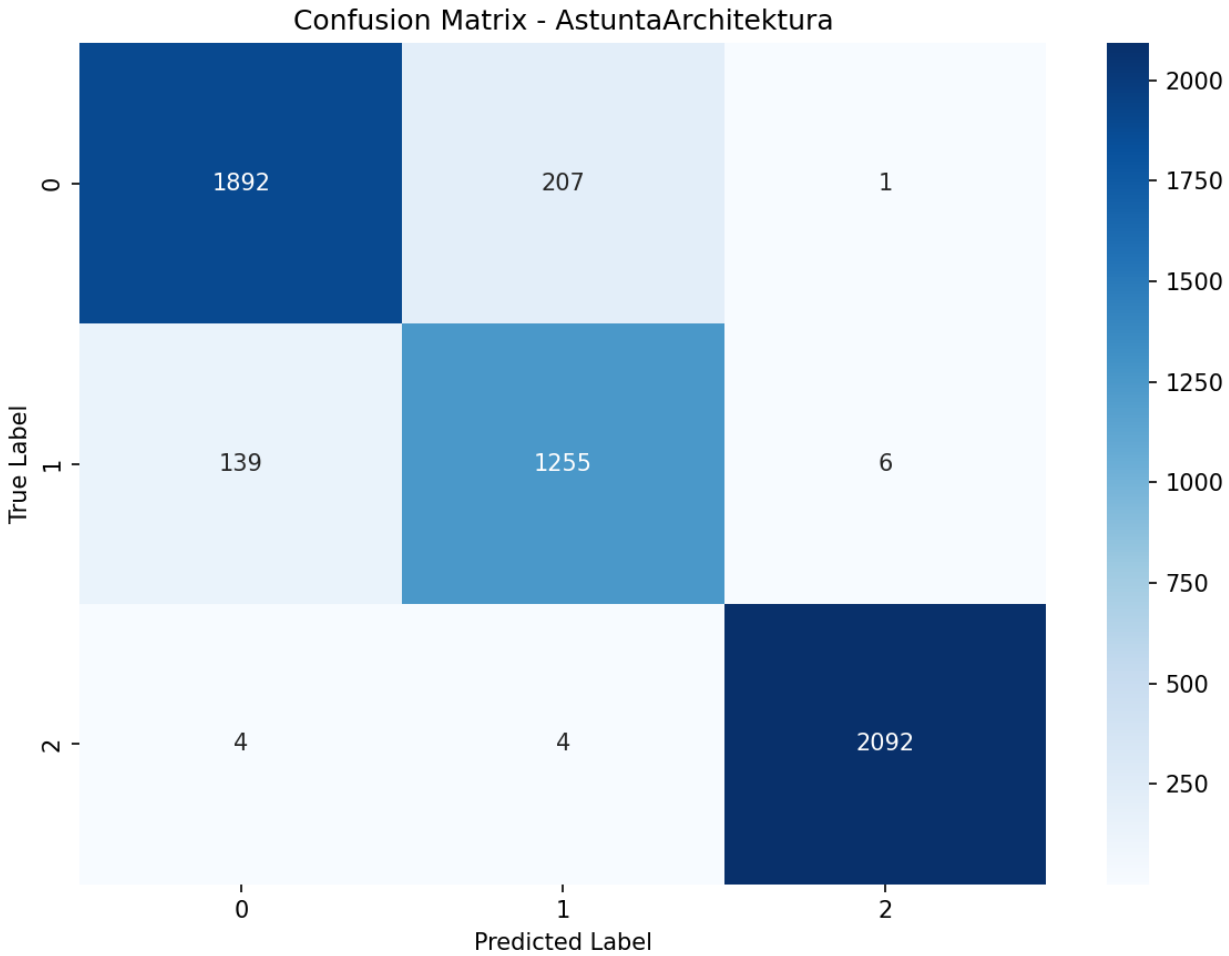
Bendras tikslumas: 93.50%

AstuntaArchitektura Du konvoliuciniai blokai su **BatchNorm** po kiekvienos konvoliucijos: Conv(1→32) → AvgPool → BN → Conv(32→64) → MaxPool → BN. Pilnai susieti sluoksniai (1600→128) taip pat turi BatchNorm normalizavimą.

Apmokymo eiga:


Train vs Validation Loss
Train Loss
Val Loss
Best epoch (13)
Loss
0.25
0.20
0.15
0.10
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0
Epoch


Train vs Validation F1
Train F1
Val F1
Best epoch (13)
F1 (macro)
0.96
0.94
0.92
0.90
0.88
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0
Epoch



Geriausia epocha: **13** – greičiausias konvergavimas iš visų architektūrų dėl partijų normalizavimo. Po 13-os epochos validavimo rodikliai stabilizuojasi.

Testavimo rezultatai:

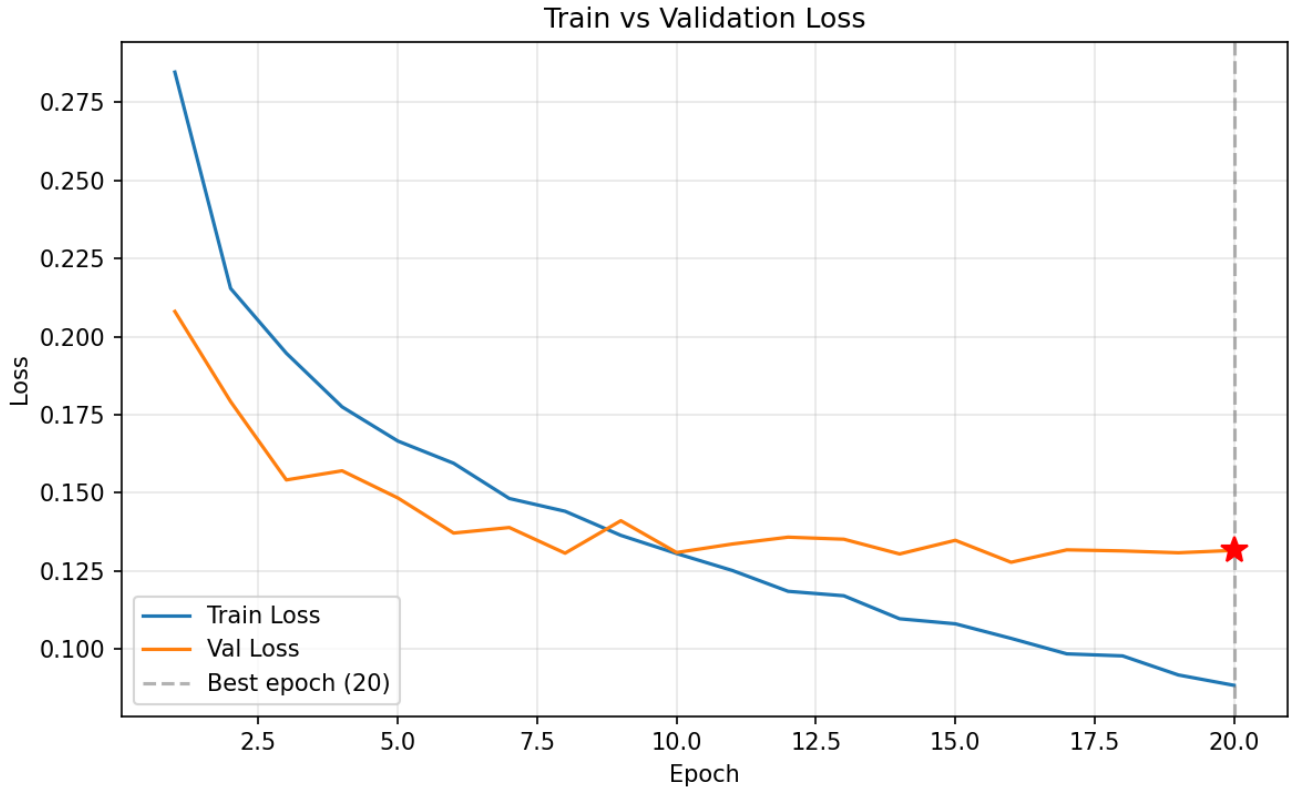
Klasė	Precision	Recall	F1
0	0.930	0.901	0.915
1	0.856	0.896	0.876
2	0.997	0.996	0.996
Makro vidurkis	**0.927**	**0.931**	**0.929**

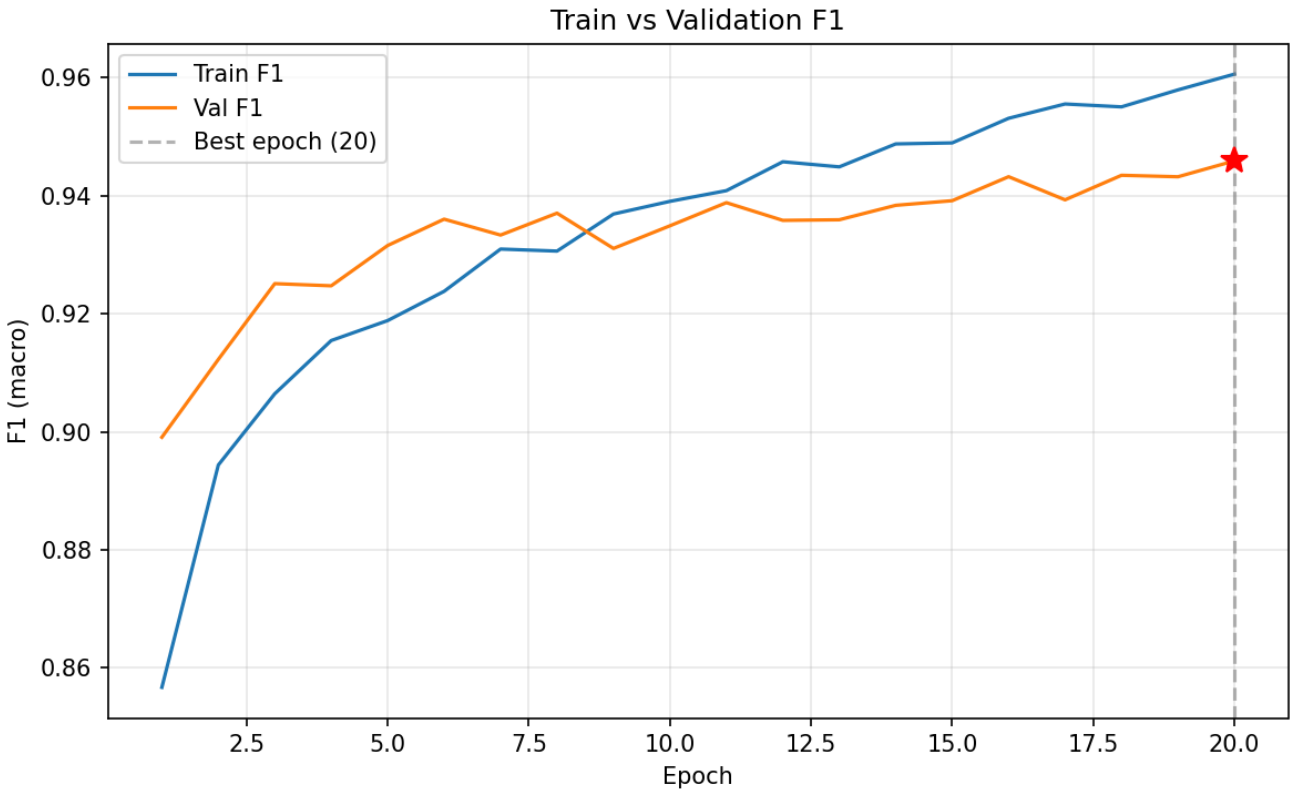
Bendras tikslumas: 93.55%

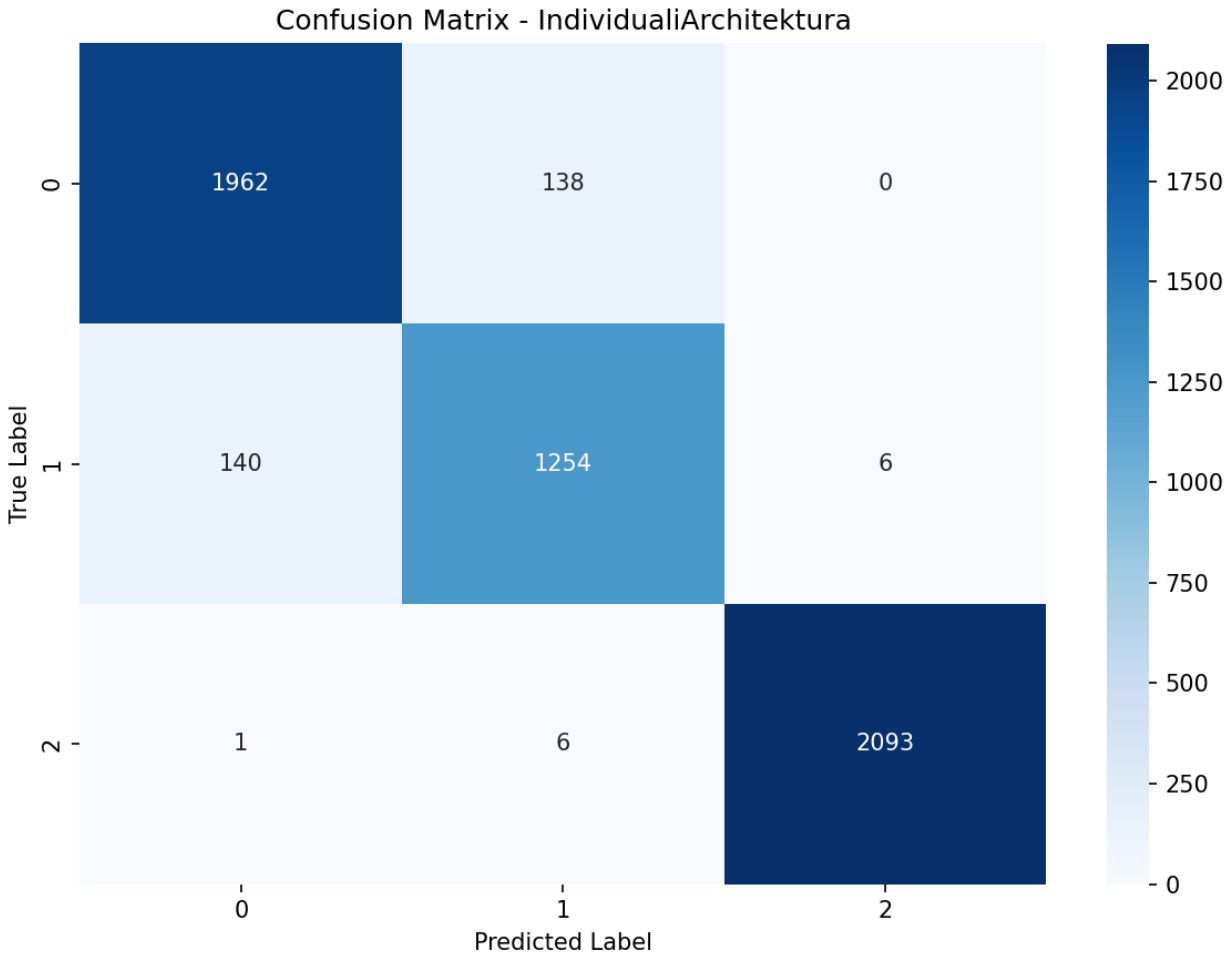
IndividualiArchitektura (pasiūlyta) Trys konvoliuciniai blokai **su padding=1**, kad išlaikytų erdvinį matmenį: Conv(1→32, pad=1) → BN → MaxPool → Dropout2d(0.1) → Conv(32→64, pad=1) → BN → MaxPool → Dropout2d(0.1) → Conv(64→128, pad=1) → BN → Dropout2d(0.1). Pilnai susieti sluoksniai: 6272→256→3, su Dropout(0.3).

Pagrindiniai sprendimų pagrindai: - **Padding** leidžia 3-iajam konvoliuciniam blokui dirbti su 7×7 feature map (o ne 3×3), todėl saugoma daugiau erdvinės informacijos. - **BatchNorm + Dropout2d** kartu sumažina persimokymą ir stabilizuoja mokymą. - Didesnis pilnai susieto sluoksnio dydis (256 neuronai) suteikia daugiau pajėgumo klasifikavimo dalyje.

Apmokymo eiga:




Train vs Validation F1
Train F1
Val F1
Best epoch (20)
F1 (macro)
0.96
0.94
0.92
0.90
0.88
0.86
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0
Epoch



Geriausia epocha: **20** – tinklas vis dar gerėjo iki paskutinės epochos, rodydamas, kad galėjo mokytis ilgiau.

Testavimo rezultatai:

Klasė	Precision	Recall	F1
0	0.933	0.934	0.934
1	0.897	0.896	0.896
2	0.997	0.997	0.997
Makro vidurkis	**0.942**	**0.942**	**0.942**

Bendras tikslumas: 94.80%

1.3 Architektūrų palyginimas

Architektūra	Tikslumas	F1 Makro	Geriausia epocha
KetvirtaArchitektura	93.02%	0.922	18

Architektūra	Tikslumas	F1 Makro	Geriausia epocha
PenktaArchitektura	93.50%	0.928	17
AstuntaArchitektura	93.55%	0.929	13
IndividualiArchitektura	**94.80%**	**0.942**	20

Išvados dėl architektūrų tinkamumo:

- Visos keturios architektūros pasiekė aukštą tikslumą (>93%), todėl sprendžiamam uždaviniui patenkinamos visos.
- Klasė 2 visose architektūrose klasifikuojama beveik tobulai (F1 > 0.994), klasė 1 – prasčiausiai, nes turi mažiau apmokymo pavyzdžių (klasių disbalansas).
- **AstuntaArchitektura** pasižymi greičiausiu konvergavimu (13 epocha) dėl BatchNorm normalizavimo – tai praktiškai svarbu, kai mokymui skiriamas ribotas laikas.
- **IndividualiArchitektura** pasiekė geriausius rezultatus (+1.8% F1 lyginant su KetvirtaArchitektura). Padding naudojimas trečiajame bloke leidžia geriau išsaugoti erdvinę informaciją, o kombinuotas BatchNorm+Dropout2d efektyviai valdo persimokymą.
- Architektūros be BatchNorm (Ketvirta, Penkta) linkusios rodyti didesnį skirtumą tarp apmokymo ir validavimo F1 rodiklių, t.y. labiau linkusios persimokymui.

1.4 Minimalios duomenų imties nustatymas (k-fold)

Naudojant **5-fold cross-validation** ir **binarinę paiešką**, nustatytas minimalus duomenų kiekis, kai IndividualiArchitektura pasiekia F1 >= 0.91. Duomenys ribojami pagal abėcėlinę paveikslėlių numeracijos tvarką (nuo 1 iki N).

Paieškos rezultatai:

Duomenų dalis	F1 (k-fold vidurkis)
10%	0.887
20%	0.902
30%	0.910
40%	0.913
35%	0.916
32.5%	0.909
33.75%	0.909
34.375%	0.911
34.0625%	0.912
33.9375%	0.912

Minimali reikalinga imtis: ~18 987 pavyzdžiai (~34% viso duomenų rinkinio)

Išvados: Tinklas pasiekia priimtiną F1 >= 0.91 naudodamas apie trečdalį duomenų. Tai rodo, kad konvoliuciniai tinklai efektyviai mokosi iš palyginti nedidelių imčių, kai duomenys yra kokybiški ir stratifikuotai parinkti.

---

2 dalis – ResNet18 su nuosavais duomenimis

2.1 Duomenų rinkinio paruošimas

Duomenys surinkti **individualiai** – 4 klasių nuotraukos:

Klasė (indeksas)	Objektas
0	battery (baterija)
1	kita (kiti objektai)
2	light_bulb (lemputė)
3	light_switch (jungiklis)

Viso: 400 nuotraukų (po ~100 kiekvienai klasei). Duomenys suskirstyti: 80% apmokymas / 20% testavimas, naudojant stratifikuotą padalijimą klasių balansui išlaikyti. Duomenys surinkti naudojant DuckDuckGo Search (DDGS) Image API ir Open Images Dataset (Google) naudojant FiftyOne.

Transformacijos: - *Bazinė*: dydžio keitimas iki 256px → centrinė apkarpoma sritis 224×224 → normalizavimas ImageNet statistika. - *Augmentacija*: papildomai – atsitiktinis horizontalus apvertimas, pasukimas ±15°, ryškumo ir kontrasto keitimas ±20%.

2.2 Modelio parinkimas

Pasirinktas **ResNet18** – gerai žinoma architektūra su liekamaisiais ryšiais (residual connections), kuri pademonstruota kaip efektyvi daugelyje vaizdų klasifikavimo uždavinių. Naudojamas mažiausias ResNet, nes duomenų nėra daug, tad milžiniškas modelis tikėtina overfittintų itin lengvai ir mokytųsi lėčiau. Galutinis klasifikavimo sluoksnis pakeistas iš 1000 klasių į 4 klases. Nagrinėjami 4 variantai:

Variantas	Apibūdinimas
pretrained + aug	ImageNet svoriai + duomenų augmentacija
pretrained + noaug	ImageNet svoriai, be augmentacijos
scratch + aug	Svoriai nuo nulio + duomenų augmentacija
scratch + noaug	Svoriai nuo nulio, be augmentacijos

2.3 Eksperimento rezultatai

Kiekvienas variantas apmokytas su 10%, 25%, 50%, 75% ir 100% duomenų. Apmokymas: 20 epochų, batch dydis 32, mokymosi greitis 0,001.

F1 Makro rodiklio palyginimas pagal duomenų kiekį:

Duomenų dalis	Pretrained + Aug	Pretrained + NoAug	Scratch + Aug	Scratch + NoAug
10%	0.867	0.250	0.208	0.650
25%	0.568	0.614	0.433	0.651
50%	0.902	**0.975**	0.747	0.729
75%	0.904	0.695	0.581	0.735
100%	**0.924**	0.754	0.735	0.776

Apmokymo eigos grafikas (visi variantai, visi duomenų kiekiai):

Figure 1: ResNet18 eksperimento rezultatai

Pastaba: Grafike kiekvienam duomenų kiekiui (10%–100%) rodoma validavimo F1 eiga 20 epochų. Kiekvienas variantas – atskira spalva; ryški linija – validavimas, blankesnė – apmokymas.

2.4 Transfer learning poveikis

Palyginimas (su vs. be ImageNet svorių), 100% duomenų:

	F1 Makro
Pretrained (ImageNet svoriai, su Aug)	0.924
Pretrained(ImageNet svoriai, be Aug)	0.754
Scratch (be svorių, su aug)	0.735
Scratch (be svorių, be aug)	0.776

Išvados: Transfer learning turi aiškią ir reikšmingą įtaką, ypač su mažomis duomenų imtimis: - Su 10% duomenų pretrained+aug modelis pasiekia F1=0.867, o scratch+aug – tik 0.208. - ImageNet svoriai suteikia galingą pradžios tašką – tinklas jau „moka” atpažinti spalvas, kraštus, tekstūras, todėl klasifikavimui su naujomis klasėmis reikia daug mažiau apmokymo duomenų. - Mokant nuo nulio su tik 400 nuotraukų ResNet18 nepasiekia transfer learning lygio – architektūra per gili ir parametrų per daug efektyviam mokymui tokiame maže duomenų kiekyje.

2.5 Duomenų augmentacijos poveikis

Palyginimas (su vs. be augmentacijos), 100% duomenų:

	F1 Makro
Pretrained + Augmentacija	0.924
Pretrained + Be augmentacijos	0.754
Scratch + Augmentacija	0.735
Scratch + Be augmentacijos	0.776

Išvados: Augmentacijos poveikis nėra vienareikšmiškai teigiamas visuose scenarijuose: - Kartu su pretrained svoriais augmentacija **pagerino** rezultatą (+0.170 F1 lyginant pretrained variante). - Mokant nuo nulio augmentacija šiek tiek **pablogino** rezultatą (0.735 vs 0.776). Tikėtina priežastis – su tokiu mažu duomenų kiekiu nuo nulio mokytas tinklas negavo pakankamai aiškių signalų, o augmentacijos sugadintas vaizdas tik trukdė. Tai nebūtinai yra tikroji priežastis, turint omenyje, kad testų kiekis nėra didelis, todėl rezultatas galėjo skirtis ir dėl atsitiktinių svorių inicializavimo. - Bendroji išvada: augmentacija efektyviausia derinant su transfer learning – ji padeda išvengti persimokymo ir pagerina generalizavimą, ypač kai duomenų imtis nedidelė.

2.6 Bendrinės išvados

1. **Transfer learning yra esminis** šiam uždaviniui: su ribotais individualiai surinktais duomenimis (400 nuotraukų) pretrained ResNet18 dramatiškai lenkia nuo nulio mokomą tinklą.
2. **Augmentacija pagerina rezultatus kartu su transfer learning** – tai rekomenduojama kombinacija klasifikavimo uždaviniams su mažomis imtimis.
3. **Duomenų kiekio įtaka** labiausiai juntama su nuo nulio momomu tinklu: scratch variantų rezultatai stipriai kinta tarp 10%–100%, o pretrained modeliai pasiekia patenkinamą tikslumą net su 10% duomenų.
4. Kai kurie rezultatai (pvz. pretrained_noaug 50% – F1=0.975) gali būti nestabilūs dėl **labai mažo testų rinkinio** (40 test pavyzdžių), todėl juos reikia vertinti atsargiai.
5. Optimalus modelis šiam uždaviniui: **ResNet18 su ImageNet svoriais ir duomenų augmentacija**, apmokytas su pilnu 100% duomenų rinkiniu (F1=0.924).

Dirbtinis intelektas (Claude Sonnet 4.6) buvo naudotas formatuojant šią ataskaitą. Visa mokymosi istorija, metrikos yra išsaugota.